Tosca concepts, taught through a Playwright twin

This guide explains the core Tosca test-design concepts and shows where each one lives in this repo.

> **Honest framing.** This is Tosca-*shaped* design implemented with Playwright. It is **not** a licensed Tosca Commander workspace. There are no `.tsu` exports, no Commander screenshots, and no connection to any Tricentis product. The concept descriptions are general and paraphrased. For authoritative definitions, use the vendor's own documentation.

1. Commander: the workspace

In Tosca: Commander is the desktop authoring environment. A shared workspace holds sections for requirements, Modules, test case design, TestCases, execution lists and results.

Twin here: the repository itself is the workspace, with one folder per section.

Workspace section	This repo
Requirements	`docs/requirements.md`
Modules	`tests/pages/`
Test case design / test data	`tests/fixtures/TestData.ts`
TestCases	`tests/{functional,integration,regression,uat}/*.spec.ts`
Reusable blocks	`tests/fixtures/ota.ts`, page-object methods
Execution lists	npm scripts in `package.json`, `.github/workflows/ci.yml`
Results	Playwright HTML/JSON report, CI artifacts, `docs/traceability.md`

2. Modules: describe a screen once

In Tosca: a Module is a technical model of an application screen or interface. You scan the screen, and each control becomes a named Module attribute with its identification properties. TestCases then reference those attributes by name and never use raw locators, so a UI change is fixed in one place.

Twin here: `tests/pages/OrderPage.ts`, a page object.

- Each control is declared once as a `Locator` field (`customer`, `zip`, `plan`, `iccid`, `submit`, …).
- Controls are identified by a stable property, `data-testid`, which plays the role of the identification property a scan would record. The attribute name is configured in `playwright.config.ts` (`testIdAttribute`).
- Specs never write selectors; they call `orderPage.zip`, `orderPage.status(id)` and so on.

3. TestCases and TestSteps

In Tosca: a TestCase is an ordered list of TestSteps. Each TestStep points at a Module and sets an *action mode* on its attributes, for example input a value, verify a value, or wait.

Twin here: each `test('TC-OTA-NNN …')` in a spec file is one TestCase.

- The steps are page-object calls; `test.step()` groups them with readable names. See `tests/uat/order-to-activate.uat.spec.ts`.
- Action modes map onto Playwright calls:

Tosca action mode (concept)	Playwright twin
Input	`locator.fill()`, `selectOption()`, `click()`
Verify	`await expect(locator).toHaveText(...)`
WaitOn	auto-waiting `expect` / locator assertions
Buffer (store a value for later)	a local variable, e.g. an order ID read from the page

4. Reusable components / TestStepBlocks

In Tosca: reusable TestStepBlocks package common step sequences, such as login or reset, so many TestCases reference one definition.

Twin here:

- `tests/fixtures/ota.ts` is a Playwright fixture. It opens the app, clears state and hands every test a ready `OrderPage`, the way a shared precondition block would.
- `OrderPage.place(order)` and `OrderPage.act(id, action)` are reusable step sequences that many tests call.

5. Test data and test case design

In Tosca: test case design separates *what* to test (attributes, instances and combinations) from the TestCase steps, so one template can run against many data rows.

Twin here: `tests/fixtures/TestData.ts`.

- `TestData.valid({ ...overrides })` gives a valid baseline and changes only the field under test.
- Data-driven loops in `tests/functional/order-entry.spec.ts` (bad ICCIDs, plan → price) are the equivalent of template instances.
- All data is fictional.

6. Execution lists and results

In Tosca: an execution list selects TestCases to run together and records each run's results.

Twin here:

- `npm run test:functional | test:integration | test:regression | test:uat` are the execution lists, one per suite (see `docs/suites.md`).
- `npm test` and the CI `playwright` job run the full regression execution list on every push.
- Results come out as a Playwright HTML report and `test-results/results.json`, both uploaded as CI artifacts.

7. Requirements and traceability

In Tosca: requirements are linked to TestCases so coverage and risk can be reported.

Twin here (BGSTM vocabulary):

- Requirement IDs `REQ-OTA-NNN` are in `docs/requirements.md`.
- Each test carries `bgstm:requirement` annotations through `Trace.req(...)` (`tests/fixtures/Trace.ts`).
- `docs/traceability.md` links each requirement to its scenarios, test cases and latest result.

What does not carry over

- **Scanning.** Tosca builds Modules by scanning the live UI; here page objects are written by hand.
- **Model-based combinatorics.** Tosca's test case design can generate combinations; here data variation is explicit loops.
- **Vendor execution infrastructure and reporting.** These are replaced by Playwright and GitHub Actions.

Teaching lab: a 10–15 minute walkthrough

Goal: see the Tosca-shaped design working end to end. You will open the mock app, run one Functional test, read the Module twin, and follow a trace row. Concepts are explained in tosca-concepts.md.

Prerequisites: Node 20+ and a clone of this repo.

```
npm ci
npx playwright install chromium
```

Step 1: open the mock app (about 3 min)

```
npm run start:mock
# open http://127.0.0.1:4173
```

1. Enter ZIP `30301` and click **Check serviceability**. You should see "Serviceable".
2. Choose **Plus 25GB**. You should see "$40.00".
3. Enter a name and ICCID `8901260000000000001`, then click **Submit order**. Row `ORD-1001` appears with status Submitted.
4. Click **Provision**, then **Activate**. Cancel becomes disabled.

Stop the server with Ctrl+C. Everything is stored in your browser; there is no backend.

Step 2: run one Functional TestCase (about 3 min)

```
npx playwright test tests/functional -g "TC-OTA-001" --headed
npx playwright show-report
```

`TC-OTA-001` submits with no customer and no plan, then verifies both errors appear and that no order was created. In the report, open the test and look at its **annotations**: `bgstm:requirement = REQ-OTA-001`.

Step 3: read the Module twin (about 4 min)

Open `tests/pages/OrderPage.ts`.

- The constructor names every control once. That is the Module and its attributes.
- `place()` and `act()` are reusable step blocks.

Now open `tests/functional/order-entry.spec.ts`. The spec contains no selectors, only calls on the Module.

Try it: change `data-testid="customer"` in `mock-app/index.html` to something else. Rerun step 2 and watch it fail, then update the one line in `OrderPage.ts` and watch it pass. That is why Modules exist. Revert both edits afterwards.

Step 4: follow one trace row (about 3 min)

Open `docs/traceability.md` and find the **REQ-OTA-005** row (no activation before provisioning).

1. `docs/requirements.md` says what REQ-OTA-005 requires.
2. `docs/scenarios.md` lists it under SC-05 and SC-06.
3. The tests are TC-OTA-009 (`tests/integration/provisioning-api.spec.ts`), TC-OTA-010 (`tests/regression/lifecycle.spec.ts`) and TC-OTA-013 (`tests/uat/`).
4. Find the annotation in code:

   ```
   grep -rn "REQ-OTA-005" tests/
   ```

5. Check the result in the latest CI run (Actions, then `ci`, then the `playwright-report` artifact).

Wrap-up questions

- Which file would you change if the plan dropdown were redesigned? (`OrderPage.ts` only.)
- Where would a new requirement REQ-OTA-009 need entries? (requirements, scenarios, a test with `Trace.req`, traceability.)
- Which execution list would you run before a release? (`npm test`, which runs every suite.)